Science and Technology Facilities Council
ISIS Neutron and Muon Source

ISIS Technical Note — DRAFT

30 October 2026
Review status: pending

PyPTC: A User Guide to the Standalone Python Interface for PTC

Draft for code, physics, and reader review
ISIS Neutron and Muon Source, Rutherford Appleton Laboratory

Keywords: ISIS RCS, PyPTC, PTC, Python interface, tracking, optics

Summary

This draft documents the standalone PyPTC Python interface in this repository. It follows the user path from building the shared library and selecting a lattice to querying optics, tracking particles, applying apertures and errors, and inspecting outputs. It distinguishes code behavior from accelerator-physics interpretation and from functions available in the underlying PTC library but not exposed by PyPTC. Claims are tied to a frozen source revision; code correspondence, physics interpretation, and reader usefulness remain pending human review.

Contents

1 Introduction and scope **3**
1.1 How to read evidence labels . . . 3
1.2 PTC literature in context . . . 3

2 Quick start **3**
2.1 Build and import . . . 4
2.2 Minimal lattice and optics call . . . 4
2.3 Expected output and next steps . . . 4

3 User path through the package **5**
3.1 Python, ABI, and parent PTC boundary . . . 5
3.2 Errors and failures . . . 5

4 Public API map **5**

5 Lattice and optics interaction **6**
5.1 One-turn maps and normal forms 7

6 Particle and bunch tracking **7**
6.1 Apertures and particle loss 8

7 Lattice edits, errors, and feature boundaries **9**
7.1 Explicitly unexposed paths 10
7.2 Examples, tests, and benchmark evidence 11

8 Evidence, limitations, and review status **11**
8.1 Unresolved interpretation items 12
8.2 Review gates 12
8.3 Reproducibility record 13

9 References **14**

1 Introduction and scope

PyPTC is the Python interface in this repository to a standalone PTC shared library. This note explains what a user can invoke through the Python package and how those calls reach the library. It is a guide to this implementation snapshot; it is not a general PTC manual, a MAD-X command guide, or a statement of ISIS operating policy. [claim:API-01]

PTC literature describes a broader polymorphic tracking and map-analysis system, including real and Taylor arithmetic and geometric lattice structures [6]. That context explains the lineage of the underlying code, but it does not prove that a feature is available in PyPTC. The exposed Python names, ABI symbols, and explicit unsupported-feature guards define the interface documented here. [claim:LIT-01]

The evidence basis is PyPTC commit `8824003c956390e921a1d469f77c247859a63258`; hashes and source roles are in `provenance/SOURCE_MANIFEST.csv`. This is an unreviewed draft. Physics meanings, sign conventions, operational suitability, and numerical acceptance criteria are not approved by this document. [claim:PROV-01]

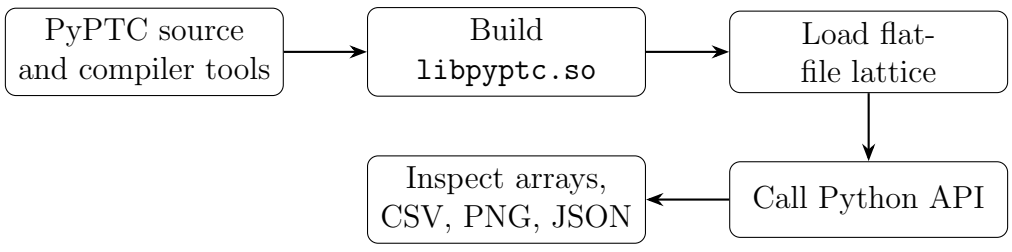
1.1 How to read evidence labels

Claims tagged `implemented` describe behavior visible in the frozen source. `derived` claims report a named computation or execution. `domain_interpretation` requires a domain authority and physics review. `unresolved` marks details not established by available evidence. The claim ledger in `provenance/CLAIMS.toml` records source symbols, line ranges, hashes, checks, and review status. [claim:PROV-02]

1.2 PTC literature in context

The original PTC report introduces PTC's symplectic-integrator, polymorphic Taylor, and fibre-bundle concepts [6]. Forest and Schmidt's PTC User's Reference Manual describes state, geometry, and tracking routines [5]. Later author papers describe FPP and the MAD-X/PTC integration [2, 3, 7, 8]. These sources provide context for the underlying library and other interfaces; they are not PyPTC conformance tests. See the curated narrative and access record in `../literature_review.md`. [claim:LIT-01]

2 Quick start



The diagram summarizes the source-level user path; each box is described in the sections that follow. [claim:API-01]

2.1 Build and import

From the repository root, run the documented local build command:

```
bash build/build_ptc.sh
python3 -c "from pyptc import PTC; print('PyPTC import OK')"
```

The build script requires `meson` and `nm`, compiles the shared library under `build_pyptc/`, checks exported symbols, and performs a ctypes load smoke check. A successful import alone does not establish that the lattice loads or that a physical calculation is validated. [claim:BUILD-01]

2.2 Minimal lattice and optics call

The default lattice constant resolves to the generated simplified flat file. A helper generates the file if it is absent, empty, or explicitly requested for regeneration. A legacy readiness file is separately named and is not the default. [claim:LATTICE-01]

```
from pyptc import PTC, DEFAULT_LATTICE

ptc = PTC()
ptc.init_lattice(DEFAULT_LATTICE)
ptc.update_twiss()
print(ptc.tunes())
rows = ptc.all_node_twiss_orbit()
print(f"rows: {len(rows)}")
```

The method sequence is directly exposed in the Python wrapper. The exact numeric values depend on the selected lattice and build and should be recorded with those inputs. This excerpt is a starter workflow, not a tolerance or acceptance test. [claim:API-02]

2.3 Expected output and next steps

The plotting examples write user-facing files under `test_outputs/`; the bare optics example writes a Twiss/orbit plot, CSV rows, and a JSON summary. The bunch example writes separate initial/final dashboards and diagnostic CSV products. Run the matching example script to regenerate outputs; copied figures in this note are frozen repository artifacts and are not generated during LaTeX compilation. [claim:EXAMPLE-01]

```
python3 examples/01_plot_bare_twiss_closed_orbit.py
python3 examples/02_track_bunch_dashboard.py
```

For installation details, lattice generation inputs, and dependencies, follow the repository README and example-specific script. [claim:BUILD-01]

3 User path through the package

The public package exports the `PTC` class, lattice helpers, parser record types, error/aperture readers, and diagnostic plotting helpers. The package is organized into API, lattice, aperture, error-table, bunch, and plotting modules. Its initializer identifies the exported subset; method behavior is traced to its defining module. [claim:API-01]

Construct `PTC(library=DEFAULT_LIBRARY)` loads the configured shared library and checks that the file exists; a missing library raises `FileNotFoundError`. [claim:API-02]

Initialize `init_lattice(path)` stores a resolved lattice path and passes it to the PTC initialization symbol. `run_script(path)` and `read_accel_table(path)` pass resolved file paths to corresponding library entry points. [claim:API-03]

Query Methods such as `tunes()`, `chromaticities()`, and `node_phase_advances()` allocate Python/ctypes outputs, call ABI symbols, check status where provided, and return Python dictionaries or NumPy arrays. [claim:API-04]

Edit Misalignment, aperture, error-table, and machine-control methods call dedicated C ABI functions from the Python wrapper. Their argument units and operational interpretation require review where not settled by code. [claim:API-05]

Track The particle-tracking methods convert Python bunch rows into the six-value PTC coordinate array, call the ring tracking entry point, and convert final values back into the package convention. [claim:TRACK-01]

3.1 Python, ABI, and parent PTC boundary

The Python wrapper configures shared-library function signatures through `ctypes`. The companion `fortran/pyptc_api.f90` contains the exposed shim implementations, which in turn call routines from the parent PTC code. The build script verifies a defined set of exported names. This note's feature claims stop at that inspected call chain; it does not provide a line-by-line account of the entire parent PTC source tree. [claim:ABI-01]

3.2 Errors and failures

The wrapper converts nonzero status values into `RuntimeError` through `_check_status`. Name-based fibre lookup requires an initialized or explicitly supplied lattice path; missing names and occurrences raise `ValueError`. Input shape guards reject particle coordinates that are not length six and bunch arrays that are not `N x 6`. [claim:FAIL-01]

4 Public API map

This inventory follows the names a Python user can call. Methods beginning with an underscore are internal helpers and are omitted. The signatures in source remain authoritative;

this summary groups methods by the interaction they support. Values and units are not inferred from names. [claim:API-06]

User task	Methods
Create/query context	`PTC(...)`; `api_level()`, `init_lattice()`, `run_script()`, `read_accel_table()`, `update_twiss()`, `machine_summary()`, `scalar()`
Read optics/orbit	`tunes()`, `chromaticities()`, `node_phase_advances()`, `node_twiss_orbit()`, `all_node_twiss_orbit()`, `task_type()`
Track particles	`track_particle_ptc()`, `track_particle_ptc_with_loss()`, `track_bunch()`, `track_bunch_with_losses()`, `particle_diagnostics()`
Edit alignment/errors	`set_misalignment()`, `set_madx_misalignment()`, `add_madx_multipole_error()`, name-based variants, `apply_misalignments()`, `apply_madx_error_table()`
Apply/query apertures	`set_aperture()`, name-based set/get, `get_aperture()`, `apply_rectangular_apertures()`, `apply_madx_aperture_file()`, `all_fibre_apertures()`, disable methods, `absolute_aperture()`, `set_absolute_aperture()`
Time-dependent controls	`set_acceleration()`, `set_ramping()`, `set_modulation()`, `set_cavity()`, orbit-state methods, `energize_lattice()`, orbit-time methods, cavity setup/power/total-path methods, AC and ramp magnet configuration

Module-level helpers include flat-file fibre parsing and name resolution; MAD-X aperture and error-table readers; JVT design aperture reader; matched Gaussian bunch helper; tune summary, CSV writers, and plotting helpers. The exported subset is listed in `pyptc/__init__.py`. Utility functions and source examples are not equivalent to one monolithic managed simulation workflow. [claim:API-01]

The time-dependent control group is present in the Python wrapper and ABI, but this guide does not assert a validated ramping, RF, or acceleration procedure for the ISIS RCS. Treat those methods as interface entry points pending a separately evidenced and physics-reviewed workflow. [claim:CONTROL-01]

5 Lattice and optics interaction

The lattice helper parses PTC flat-file fibre blocks and exposes one-based fibre indices, element names, lengths, and source line spans. Name resolution first looks for an exact element-name match, then supports a parent dipole name expanding to numbered sliced dipole records. This is a parser contract for the flat-file form, not a general MAD-X sequence parser. [claim:LATTICE-02]

After lattice initialization and Twiss update, users can query tunes, chromaticities, phase advances, and per-node Twiss/orbit data through `PTC`. The code returns values and field names; physical coordinate meaning, unit conventions, and validity ranges are separate review questions. Do not infer them from names alone. [claim:API-04]

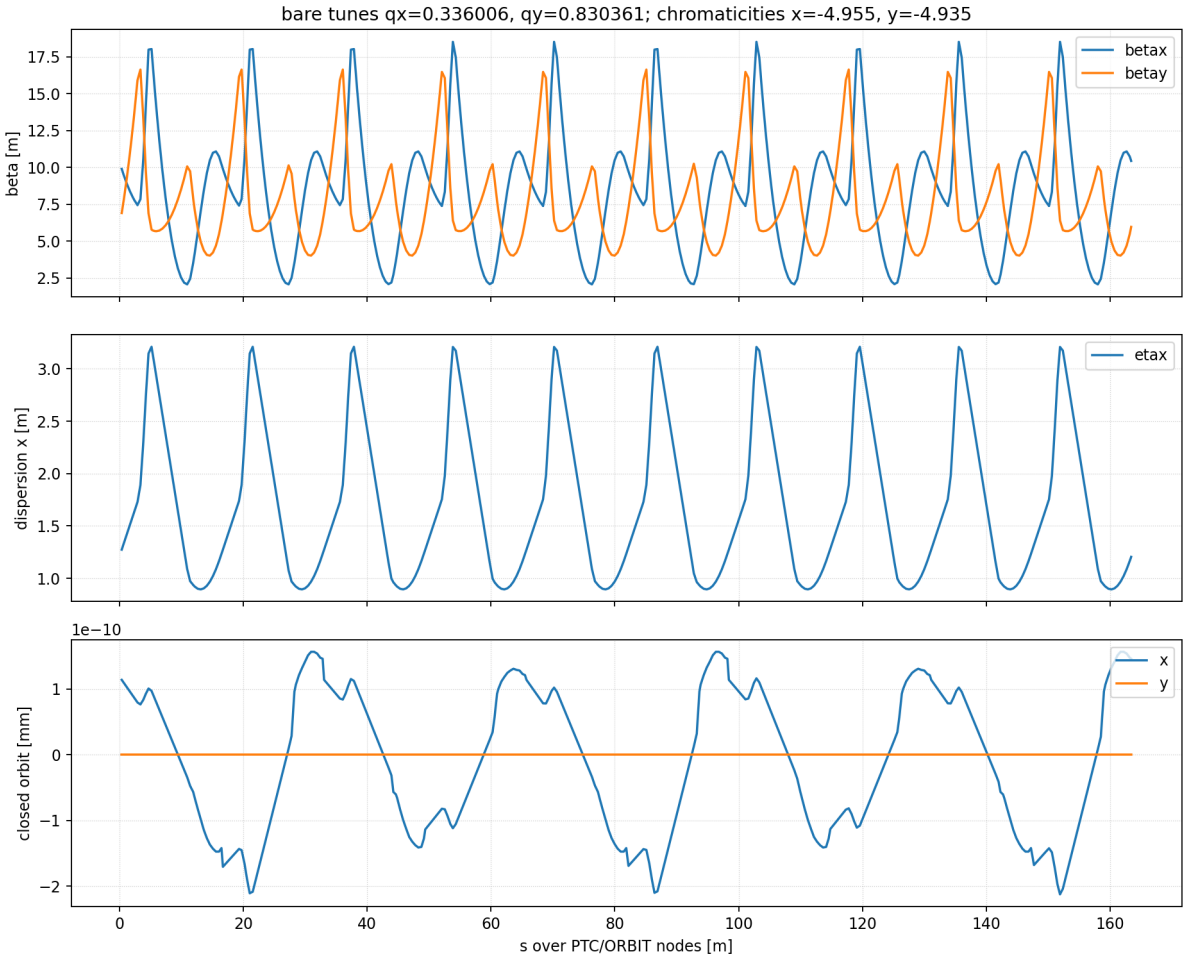


Figure 1: Repository optics diagnostic generated by the bare optics example. The copied pixels are unchanged. It illustrates the plotted fields available from this example; it is not a comparison to a reference model or physics acceptance. [claim:PROV-03]

5.1 One-turn maps and normal forms

`one_turn_map()` and `normal_form()` both raise the not-exposed exception. [claim:LIMIT-01]

6 Particle and bunch tracking

The package accepts bunch input as a two-dimensional six-column array. `track_bunch_with_losses()` copies and validates the array, converts each row into the PTC six-coordinate

representation, tracks each particle, then converts the final values back. The `particle_diagnostics()` routine records turn-by-turn transverse phase and survival information and returns per-particle diagnostic rows. These are implementation descriptions; the physical meaning and units of every coordinate require physics review. [claim:TRACK-01]

For tracking without the loss-result wrapper, `track_bunch()` also requires an `N x 6` array and rejects other shapes with `ValueError`. It copies the input, walks the machine nodes for each requested turn, updates the synchronous reference at each node, and returns either the final array or one copied snapshot per turn when `record_turns=True`. This method's Python-to-PTC coordinate conversion is likewise implementation evidence; coordinate units and physical interpretation remain pending. [claim:TRACK-03] [claim:TRACK-02]

The standard bunch example creates its input with the PyParticleBunch helper, tracks a fixed-size bunch for a stated turn count, and writes before/after diagnostic dashboards and CSV files. The outputs show an example run only; they do not prove model validity for other lattices or operating conditions. [claim:EXAMPLE-02]

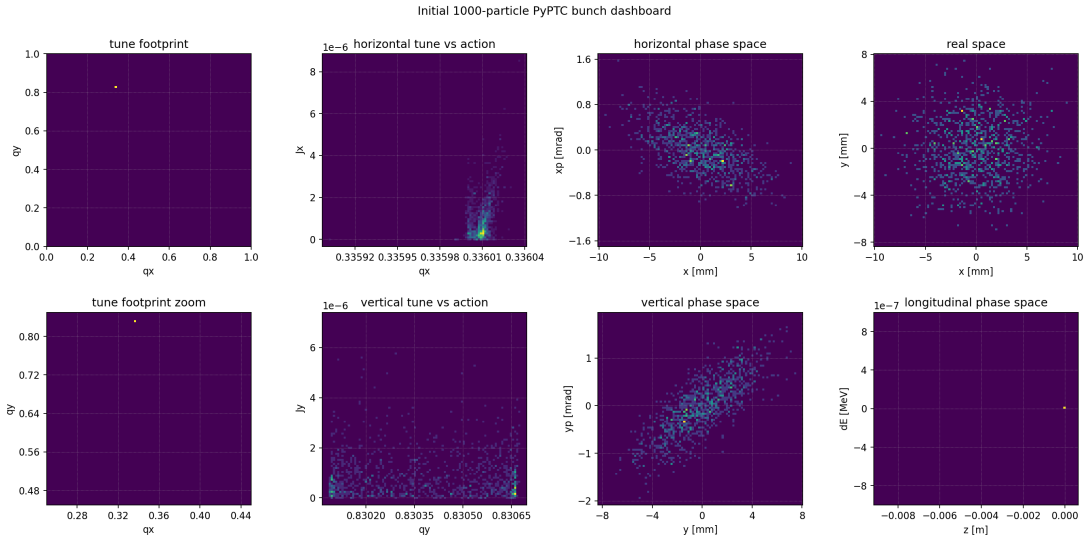


Figure 2: Initial bunch dashboard copied from the standard bunch example output. It records the available diagnostic view before tracking; the associated output directory contains the run summary and diagnostic data. [claim:PROV-03]

6.1 Apertures and particle loss

The aperture helpers read rectangular aperture assignments or TFS records, and the JVT CSV reader divides its semi-aperture columns by 1000 before constructing records. The API can apply, query, disable, and set absolute apertures; tracking returns loss flags and loss turn/position fields. The dimensions and operational meaning of input columns should be confirmed by the aperture data owner. [claim:AP-01]

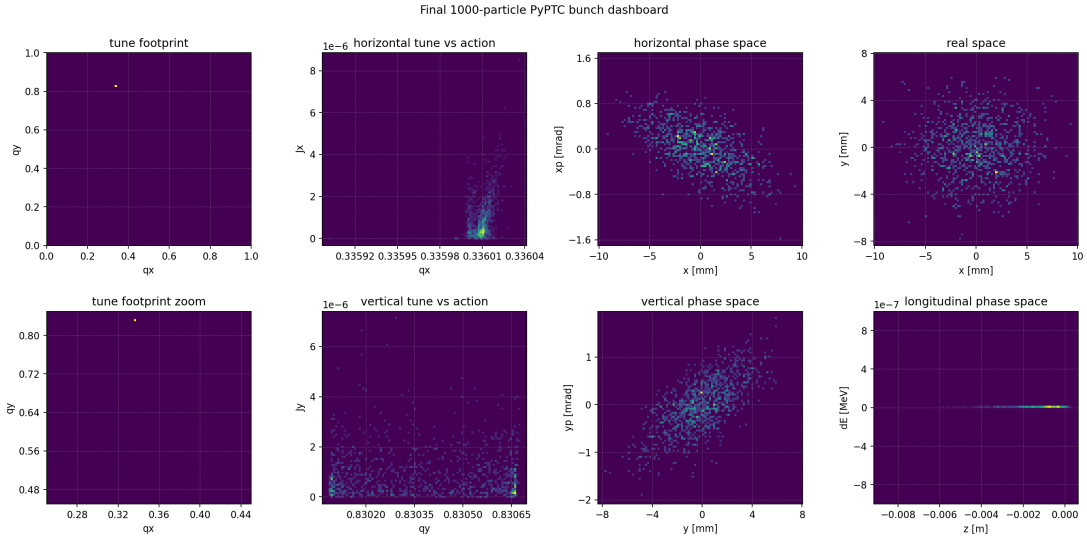


Figure 3: Final bunch dashboard from the same example output. Its side-by-side interpretation is limited to this recorded run and the generated input bunch. [claim:PROV-03]

7 Lattice edits, errors, and feature boundaries

The Python interface exposes element misalignment operations and a parser/application path for MAD-X EFIELD records. The error reader recognizes alignment columns and integrated normal/skew multipole columns through the documented order range. It retains other numeric columns so the caller can reject unsupported nonzero fields instead of silently dropping them. The exact field semantics and units need domain review against the relevant MAD-X/PTC contract. [claim:ERR-01]

The aperture readers and error-table readers are Python-side parsers. They should be treated as converters with explicit supported fields, not as complete MAD-X parsers. Their tests are parser checks; side-by-side example scripts are separate workflow evidence. [claim:API-05]

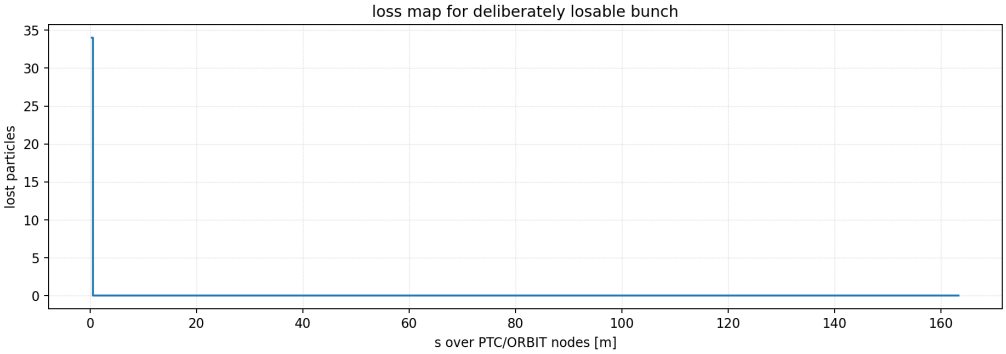
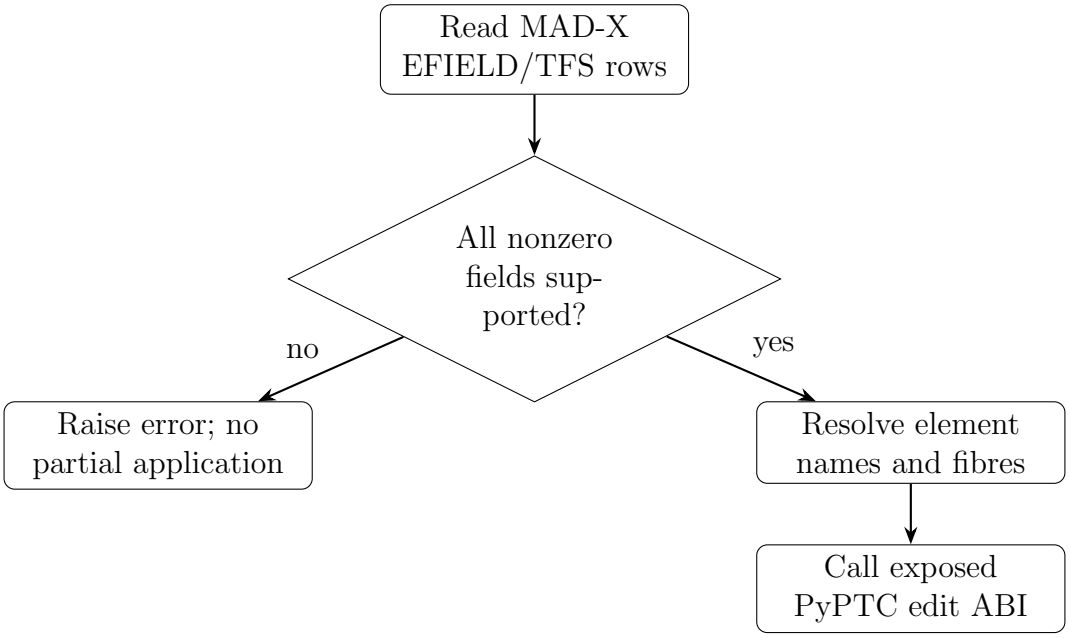


Figure 4: Loss-map image from the aperture example output. It is an example visualization from the stored run and does not establish a validated physical aperture model. [claim:PROV-03] [claim:PROV-04]



This diagram represents the wrapper's guarded error-table path. The named interpretation of individual MAD-X fields remains a separate review item. [claim:ERR-01]

7.1 Explicitly unexposed paths

The optics calls `one_turn_map()` and `normal_form()` raise a dedicated not-exposed exception. They do not fall back to another calculation. [claim:LIMIT-01]

The normalized-aperture-scan and tune-smear helpers raise the same not-exposed exception. Other PTC features may exist in the underlying library; this guide claims availability only where the repository provides an exposed wrapper path. [claim:LIMIT-02]

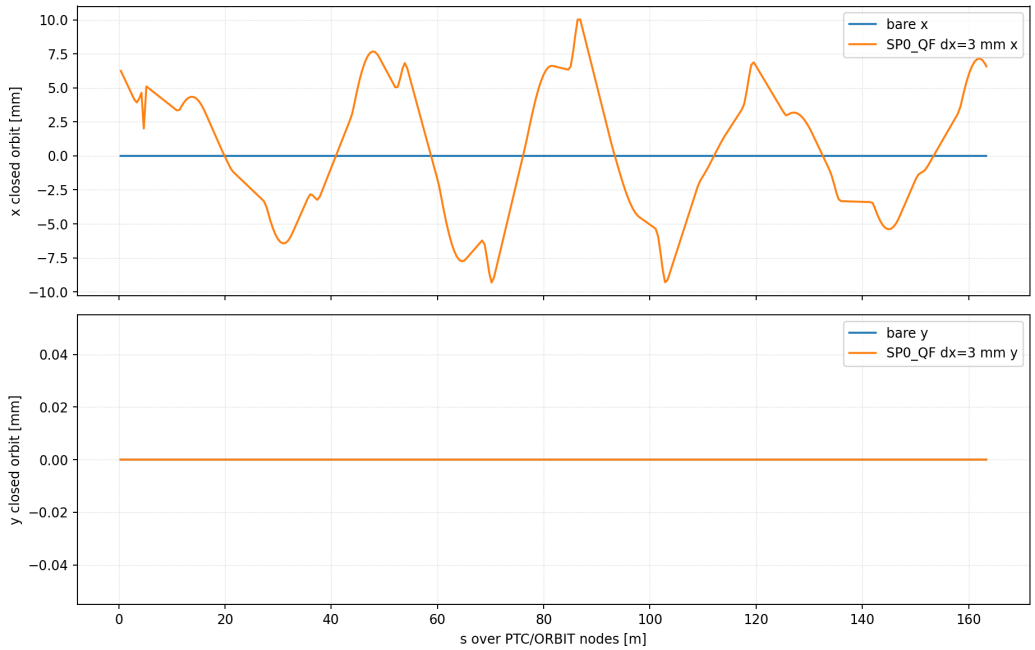


Figure 5: Single-element misalignment orbit-response plot from the repository example. The example applies a recorded horizontal displacement and compares returned orbit rows. It demonstrates a workflow path, not a machine-calibrated response prediction. [claim:PROV-03] [claim:PROV-04]

7.2 Examples, tests, and benchmark evidence

The `examples/` scripts are runnable demonstrations with user-facing plots and output summaries. `tests/` contains focused parser and helper checks and separate benchmark-oriented checks. Neither source class should be described as the public API itself. In particular, a MAD-X comparison is evidence for its named case and inputs, not automatic proof of all PyPTC physics behavior. [claim:EXAMPLE-01]

8 Evidence, limitations, and review status

The figures included in this note are byte-identical copies of repository outputs. The figure manifest gives each source path, hash, command or run context, selection rationale, and the limited claim it supports. Source code hashes, the claim ledger, and literature PDF access status are maintained alongside this note. [claim:PROV-03]

The literature describes broad PTC capabilities, but the current PyPTC wrapper exposes a narrower and versioned subset. This note does not establish validation for an ISIS machine model, acceptance tolerances, or operational use. Existing admission and MAD-X optics benchmark work has its own feedback, ledger, and attempt evidence and must be reviewed through those campaign records. [claim:LIT-01]

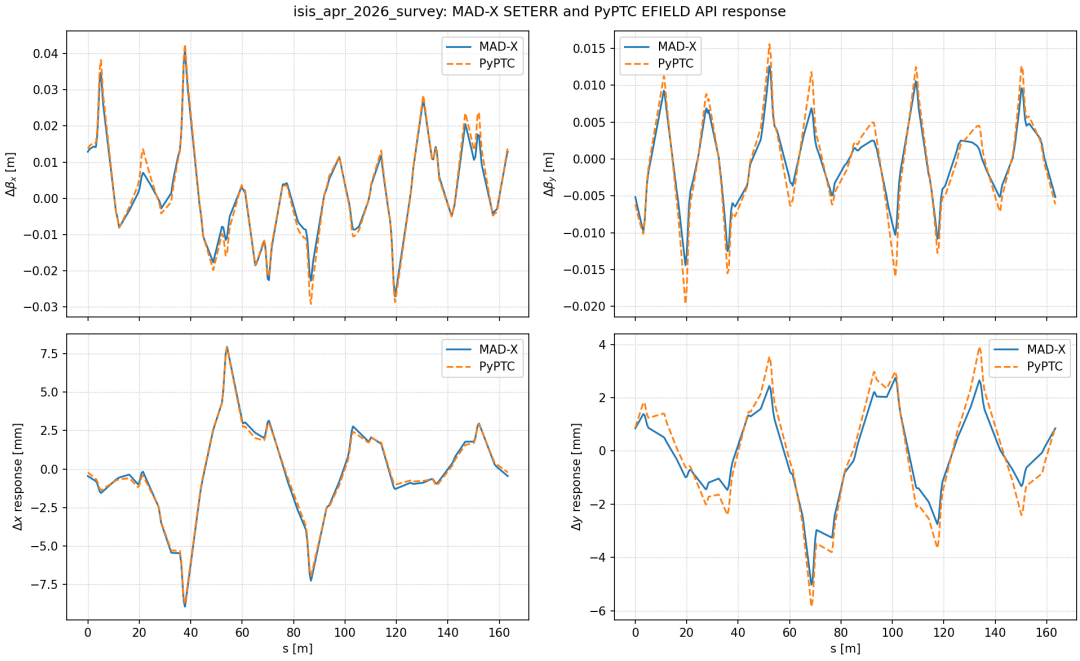


Figure 6: Stored MAD-X/PyPTC EFIELD side-by-side example for the April 2026 survey table. The companion report records both engines, input and library hashes, and states that the comparison is not tolerance-based acceptance. [claim:PROV-03] [claim:ERR-02] [claim:PROV-04]

8.1 Unresolved interpretation items

The following items require named human review before this note can be treated as approved technical guidance:

- Python bunch coordinates versus the internal PTC coordinate slots, including units, signs, and longitudinal scaling. [claim:TRACK-02]
- The meaning and unit convention of returned Twiss, orbit, chromaticity, and phase fields. [claim:PHYS-01]
- MAD-X EFIELD alignment/multipole unit and sign mapping at the PyPTC boundary. [claim:ERR-02]
- Which repository examples and benchmark cases are suitable as approved golden cases. [claim:PROV-04]

8.2 Review gates

Code correspondence: pending. Physics interpretation: pending. Reader usefulness: pending. No agent has marked any claim reviewed. A future reviewer should be named in `CLAIMS.toml`, answer the acceptance questions in the evidence skill, and record the review date and disposition. [claim:PROV-02]

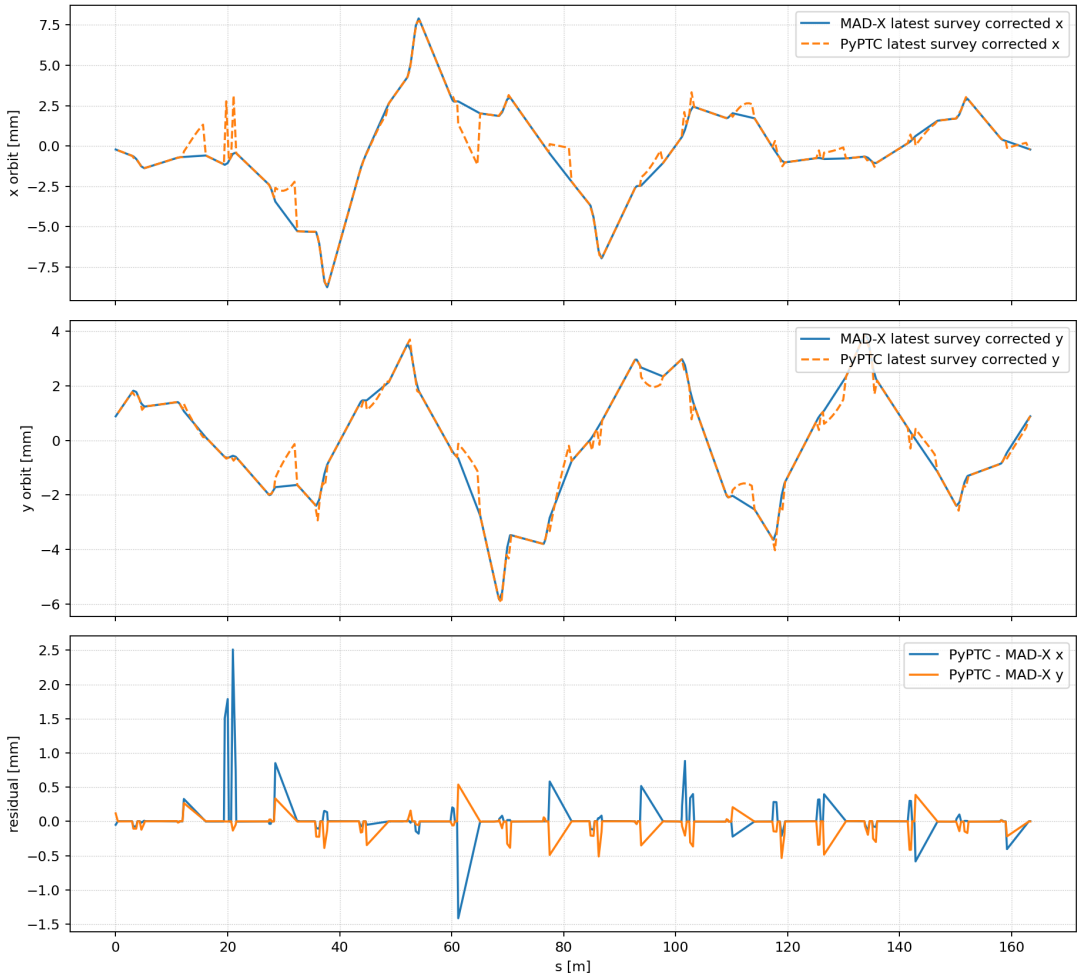


Figure 7: Stored full error-table closed-orbit comparison output. This figure belongs to the repository's specific comparison example and its stated table/lattice inputs; it does not define an acceptance tolerance. [claim:PROV-03] [claim:PROV-04]

8.3 Reproducibility record

The source manifest records the checked-out commit and hashes. The quick-start build command is `bash build/build_ptc.sh`; the note itself is built from this directory with `pdflatex` and `bibtex`. The execution log, exact commands, result, PDF hash, and page-inspection notes are recorded in `PLAN.md`. Drafting used an AI assistant with human review pending. [claim:PROV-01]

9 References

References

[1] CERN. *MAD-X PTC Reference Manual*, 2006. Historical manual cited in later PTC literature; current download not verified.

[2] E. Forest, Y. Nogiwa, and F. Schmidt. The fpp and ptc libraries. In *Proceedings of the 8th International Computational Accelerator Physics Conference (ICAP 2006)*, pages 17–21, Chamonix, France, 2006. Paper MOM1MP02.

[3] E. Forest, Y. Nogiwa, and F. Schmidt. The fpp documentation. In *Proceedings of the 8th International Computational Accelerator Physics Conference (ICAP 2006)*, Chamonix, France, 2006. Paper WEPPP04.

[4] Étienne Forest. *From Tracking Code to Analysis: Generalised Courant-Snyder Theory for Any Accelerator Model*. Springer Japan, 2016.

[5] Étienne Forest and Frank Schmidt. *PTC User's Reference Manual*. CERN, BE Department, Geneva, Switzerland, October 2010. PDF title page dated October 15, 2010; 68 pages.

[6] Étienne Forest, Frank Schmidt, and Eric McIntosh. Introduction to the polymorphic tracking code: Fibre bundles, polymorphic taylor types and "exact tracking". Technical Report CERN-SL-2002-044 (AP); KEK-Report 2002-3, CERN and KEK, July 2002. Official PDF endpoint returned an anti-bot HTML page during retrieval on 2026-10-30.

[7] Frank Schmidt. MAD-X PTC Integration. In *Proceedings of the 21st IEEE Particle Accelerator Conference (PAC 2005)*, pages 1272–1274, Knoxville, Tennessee, USA, 2005. Paper MPPE012.

[8] P. K. Skowronski, F. Schmidt, and E. Forest. Advances in mad-x using ptc. In *Proceedings of the 22nd IEEE Particle Accelerator Conference (PAC 2007)*, pages 3381–3383, Albuquerque, New Mexico, USA, 2007. Paper THPAN070.